

1. Baza danych

XML jako baza danych do rozproszonego systemu aukcyjnego to raczej pułapka niż skrót.

Do takiego projektu lepiej:

- **PostgreSQL** albo **SQL Server**
- ORM: **Entity Framework Core**

Dlaczego?

- łatwiejsze relacje User -> Auction -> Bid
- prostsze zapytania
- sensowna obsługa współbieżności
- mniej ręcznego dżubania

XML można co najwyżej użyć do importu/eksportu, ale nie jako główną bazę.

2. Frontend

W roadmapie jest Java (React/Angular)?, co jest trochę potworkiem Frankensteina, bo:

- **React** i **Angular** to frontend JS/TS
- **Java** nie ma tu nic do gadania

Skoro macie .NET w backendzie, to najczyściej będzie:

- **Blazor WebAssembly Hosted**
albo
- **React + ASP.NET Core API**

Jeśli chcecie mniej chaosu i łatwiejszą integrację:

bierzcie Blazor.

3. Security

W modelu User nie trzymajcie pola Password jako zwykłego hasła.

Powinno być coś w stylu:

- PasswordHash

Nigdy surowe hasło. Nigdy. Ani na chwilę. Ani “na razie”. To właśnie robi największy bałagan.

Proponowany sensowny podział pracy dla 4 osób

1. Magdalena - Database & Repository Architect

Działka:

- zaprojektowanie bazy danych
- stworzenie encji:
 - User
 - Auction
 - Bid
- relacje między tabelami
- migracje EF Core
- repozytoria lub warstwa dostępu do danych

Dodatkowo proponuję:

- przygotować AppDbContext
- seed testowych danych
- ustalić typy pól, np.:
 - ceny: decimal
 - daty: DateTime
 - ID: int lub Guid

2. Krzysztof - Business Logic Specialist

Działka:

- logika aukcji
- tworzenie aukcji
- składanie ofert
- walidacje biznesowe, np.:
 - oferta musi być wyższa od obecnej ceny
 - aukcja nie może być zakończona
 - sprzedawca nie może licytować własnej aukcji
- aktualizacja CurrentPrice
- wyliczanie statusu aukcji

To jest serce systemu. Jak to siądzie, reszta będzie miała na czym stać.

3. Marcin - API & Security Engineer

Działka:

- kontrolery REST API
- DTO
- mapowanie request/response
- autoryzacja i uwierzytelnianie
- JWT
- zabezpieczenie endpointów
- obsługa błędów HTTP

Przykład:

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auctions
- GET /api/auctions/{id}
- POST /api/auctions/{id}/bids

4. Katarzyna - Frontend Developer

Działka:

- interfejs użytkownika
- widok listy aukcji
- szczegóły aukcji
- formularz tworzenia aukcji
- formularz licytacji
- logowanie/rejestracja
- połączenie z API

Jeśli bierzecie **Blazor**, to super, bo frontend i backend będą grały w jednej orkiestrze, a nie w dwóch różnych weselach.

Jak ja bym ustawił strukturę projektu

Backend

- Controllers
- Models
- DTOs
- Services
- Repositories
- Data
- Mappings
- Middleware

Frontend

- Pages
- Components
- Services
- Models

Minimalne endpointy na start

Auth

- POST /api/auth/register
- POST /api/auth/login

Auctions

- GET /api/auctions
- GET /api/auctions/{id}
- POST /api/auctions
- PUT /api/auctions/{id}
- DELETE /api/auctions/{id}

Bids

- POST /api/auctions/{id}/bids
- GET /api/auctions/{id}/bids

Proponowane modele po lekkim ogarnięciu

User

- Id
- Username
- Email
- PasswordHash

Auction

- Id
- Title
- Description
- Category
- StartingPrice
- CurrentPrice
- StartTime
- EndTime
- SellerId

Bid

- Id
- AuctionId
- BuyerId
- Amount
- Timestamp

Kolejność prac, żeby nie robić wszystkiego naraz

Etap 1

- ustalenie technologii
- baza danych
- encje
- migracje
- szkielet projektu

Etap 2

- rejestracja i logowanie
- JWT
- podstawowe endpointy aukcji

Etap 3

- logika licytacji
- walidacje
- testy

Etap 4

- frontend
- integracja z backendem
- dopracowanie UI

Moja konkretna rekomendacja technologiczna

Najrozsądniejszy zestaw dla was:

- **ASP.NET Core Web API**
- **Entity Framework Core**
- **SQL Server** lub **PostgreSQL**
- **JWT Authentication**
- **Blazor WebAssembly Hosted**

To będzie spójne, sensowne i obroni się na projekcie bez kombinowania.